Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра интеллектуальных информационных технологий

Лабораторная работа №1

По дисциплине «Интеллектуальный анализ данных»

Тема: «PCA»

Выполнил:

Студент 4 курса

Факультета ЭИС

Группы ИИ-25

Милевский А.Д.

Проверил:

Андренко К.В.

Брест 2026

Цель работы: научиться применять метод PCA для осуществления визуализации данных.

Общее задание

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент (двумя способами: 1. вручную через использование numpy.linalg.eig для вычисления собственных значений и собственных векторов и 2. с помощью sklearn.decomposition.PCA для непосредственного применения метода PCA – два независимых варианта решения);

2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки matplotlib, обозначая экземпляры разных классов с использованием разных цветовых маркеров;

3. Используя собственные значения, рассчитанные на этапе 1, вычислить потери, связанные с преобразованием по методу PCA. Сделать выводы;

4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Задание по вариантам

№ варианта	Выборка	Класс
9	heart+failure+clinical+records.zip	death_event

Ход работы

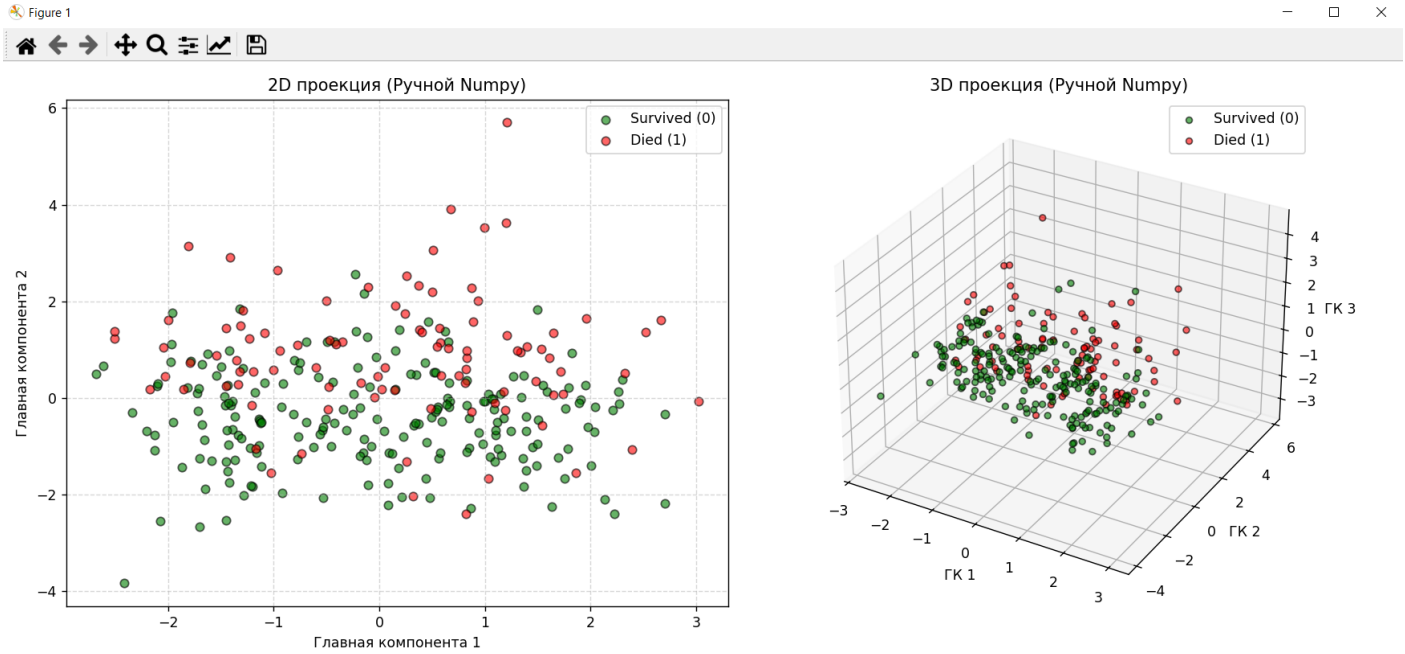
Код программы:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

df = pd.read_csv('heart_failure_clinical_records_dataset.csv')
X = df.drop('DEATH_EVENT', axis=1)
y = df['DEATH_EVENT']
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print(f"Исходная размерность данных: {X_scaled.shape}")
print(f"Уникальные классы: {np.unique(y)}\n")
cov_matrix = np.cov(X_scaled, rowvar=False)
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]
X_pca_manual_2d = X_scaled.dot(eigenvectors[:, :2])
X_pca_manual_3d = X_scaled.dot(eigenvectors[:, :3])
print("--- Ручной PCA (Numpy) ---")
print(f"Первые 3 собственных значения: {eigenvalues[:3].round(4)}")
pca_2d = PCA(n_components=2)
X_pca_sklearn_2d = pca_2d.fit_transform(X_scaled)
pca_3d = PCA(n_components=3)
X_pca_sklearn_3d = pca_3d.fit_transform(X_scaled)
print("\n--- PCA (Sklearn) ---")
print(f"Объясненная дисперсия (2 компоненты): {pca_2d.explained_variance_ratio_.round(4)}")
fig = plt.figure(figsize=(14, 6))
ax1 = fig.add_subplot(1, 2, 1)
for cls, color, label in zip([0, 1], ['green', 'red'], ['Survived (0)', 'Died (1)']):
    mask = (y == cls)
    ax1.scatter(X_pca_manual_2d[mask, 0], X_pca_manual_2d[mask, 1],
                c=color, label=label, alpha=0.6, edgecolors='k')
ax1.set_title('2D проекция (Ручной Numpy)')
ax1.set_xlabel('Главная компонента 1')
ax1.set_ylabel('Главная компонента 2')
ax1.legend()
ax1.grid(True, linestyle='--', alpha=0.5)
ax2 = fig.add_subplot(1, 2, 2, projection='3d')
for cls, color, label in zip([0, 1], ['green', 'red'], ['Survived (0)', 'Died (1)']):
    mask = (y == cls)
    ax2.scatter(X_pca_manual_3d[mask, 0], X_pca_manual_3d[mask, 1], X_pca_manual_3d[mask, 2],
                c=color, label=label, alpha=0.6, edgecolors='k')
ax2.set_title('3D проекция (Ручной Numpy)')
ax2.set_xlabel('ГК 1')
ax2.set_ylabel('ГК 2')
```

```
ax2.set_zlabel('ГК 3')
ax2.legend()
plt.tight_layout()
plt.show()
total_var = sum(eigenvalues)
loss_2d = total_var - sum(eigenvalues[:2])
loss_2d_percent = (loss_2d / total_var) * 100
loss_3d = total_var - sum(eigenvalues[:3])
loss_3d_percent = (loss_3d / total_var) * 100
print("\n--- Анализ потерь ---")
print(f"Потери при 2D проекции: {loss_2d:.4f} ({loss_2d_percent:.2f}%)")
print(f"Потери при 3D проекции: {loss_3d:.4f} ({loss_3d_percent:.2f}%)")
print("\n--- Выводы ---")
print("1. Потери при переходе к 2 и 3 компонентам довольно высоки.")
print("2. Это связано с тем, что признаки в датасете слабо коррелируют между собой.")
print("3. На графиках видно сильное наложение классов, поэтому PCA здесь лучше использовать")
print("   только для визуализации, а не для классификации.")
```

Графики:



Вывод программы:

```
Run  ИАД1

C:\Users\artem\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\artem\PyCharmMiscProject\ИАД1.py
Исходная размерность данных: (299, 12)
Уникальные классы: [0 1]

--- Ручной PCA (Numpy) ---
Первые 3 собственных значения: [1.6685 1.5844 1.2724]

--- PCA (Sklearn) ---
Объясненная дисперсия (2 компоненты): [0.1386 0.1316]

--- Анализ потерь ---
Потери при 2D проекции: 8.7873 (72.98%)
Потери при 3D проекции: 7.5149 (62.41%)

--- Выводы ---
1. Потери при переходе к 2 и 3 компонентам довольно высоки.
2. Это связано с тем, что признаки в датасете слабо коррелируют между собой.
3. На графиках видно сильное наложение классов, поэтому PCA здесь лучше использовать
   только для визуализации, а не для классификации.

Process finished with exit code 0
```

Вывод: научился применять метод PCA для осуществления визуализации данных.